# LAPORAN PRAKTIKUM
# ALGORITMA & STRUKTUR DATA
# MODUL 2

## STACK DAN QUEUE

**Oleh:**

**Salma Nur Lathifah NIM. 2510817120013**

**PROGRAM STUDI TEKNOLOGI INFORMASI**

**FAKULTAS TEKNIK**

**UNIVERSITAS LAMBUNG MANGKURAT**

**APRIL**

**2026**

# LEMBAR PENGESAHAN
# LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
# MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack dan Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Salma Nur Lathifah
NIM : 2510817120013

| | |
|---|---|
| Menyetujui, | Mengetahui, |
| Asisten Praktikum | Dosen Penanggung Jawab Praktikum |
| | |
| Muhammad Fauzan Ahsani | Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D |
| NIM. 2310817310009 | NIP. 198606132015041001 |

# DAFTAR ISI

LEMBAR PENGESAHAN ..... i
DAFTAR ISI ..... ii
DAFTAR GAMBAR ..... iii
DAFTAR TABEL ..... iv
SOAL 1: Implementasi Stack ..... 1
A. Source Code ..... 1
B. Pembahasan ..... 2
SOAL 2: Si Palui dan Mesin Hitung Aceh ..... 6
A. Source Code ..... 7
B. Output Program ..... 8
C. Pembahasan ..... 9
SOAL 3: Implementasi Queue ..... 12
A. Source Code ..... 12
B. Pembahasan ..... 14
SOAL 4: Si Palui dan Deretan Angka ..... 18
A. Source Code ..... 19
B. Output Program ..... 20
C. Pembahasan ..... 21
TAUTAN GIT ..... 23

# DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 2A ..... 8
Gambar 2. Screenshot Hasil Jawaban Soal 2B ..... 9
Gambar 3. Screenshot Hasil Jawaban Soal 2C ..... 9
Gambar 4. Screenshot Hasil Jawaban Soal 4A ..... 20
Gambar 5. Screenshot Hasil Jawaban Soal 4B ..... 20
Gambar 6. Screenshot Hasil Jawaban Soal 4C ..... 20

# DAFTAR TABEL

Tabel 1 Source Code stack.cpp..... 1
Tabel 2 Source Code prob1.cpp ..... 7
Tabel 3 Source Code queue.cpp ..... 12
Tabel 4 Source Code prob2.cpp ..... 19

# SOAL 1: Implementasi Stack

Pada file `stack.h` yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file `stack.cpp`. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan `throw`.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

## A. Source Code

*Tabel 1 Source Code stack.cpp*

```
#include "stack.h"
#include <stdexcept>

void init(Stack* s) {
    s->top = nullptr;
}

bool isEmpty(const Stack* s) {
    return s->top == nullptr;
}

bool isFull(const Stack* s) {
    if (isEmpty(s)) return false;
    return s->top == &(s->data[MAX - 1]);
}

void push(Stack* s, int value) {
    if   (isFull(s))   throw   std::overflow_error("Stack
Penuh");
```

```
19
20      if (isEmpty(s)) {
21          s->top = &(s->data[0]);
22      } else {
23          s->top++;
24      }
25      *(s->top) = value;
26  }
27
28  void pop(Stack* s) {
29      if  (isEmpty(s))  throw  std::underflow_error("Stack
    Kosong");
30
31      if (s->top == &(s->data[0])) {
32          s->top = nullptr;
33      } else {
34          s->top--;
35      }
36  }
37
38  int peek(const Stack* s) {
39      if  (isEmpty(s))  throw  std::underflow_error("Stack
    Kosong");
40      return *(s->top);
41  }
```

**B. Pembahasan**

Pada baris [1], `#include "stack.h"` program mengimpor file header yang berisi definisi struktur Stack dan deklarasi seluruh fungsi yang akan diimplementasikan. Hal ini dilakukan untuk memastikan konsistensi antara deklarasi dan implementasi, sehingga tidak terjadi ketidakcookan tipe data atau fungsi. Secara mekanisme, preprocessor akam menyalin isi file header ke dalam file ini sebelum proses kompilasi, sehingga compiler dapat

mengenali struktur Stack beserta anggotanya. Pada baris [2], `#include <stdexcept>` digunakan untuk mengakses kelas exception seperti `std::overflow_error` dan `std::underflow_error`. Hal ini penting karena struktur data berbasis array memiliki batas kapasitas tetap, sehingga diperlukan mekanisme untuk menangani kondisi pelanggaran batas. Cara kerjanya adalah ketika kondisi error terpenuhi, program akan melempar exception menggunakan `throw`, yang menghentikan eksekusi normal dan memberikan sinyal kesalahan. Selanjutnya pada implementasi ini, digunakan tanda kurung kurawal `{}` untuk membatasi blok kode dalam fungsi maupun struktur kontrol seperti percabangan dan perulangan. Pada baris [4], `void init(Stack* s) {` mendefinisikan fungsi inisialisasi `stack` dengan parameter berupa pointer ke struktur stack. Penggunaan pointer bertujuan agar perubahan terhadap top langsung memodifikasi objek asli di memori, bukan salinannya. Baris [5], terdapat `s->top = nullptr;` untuk menginisialisasi pointer top agar tidak menunjuk ke alamat mana pun yang menandakan bahwa stack kosong. Penggunaan `nullptr` lebih aman karena menyatakan bahwa pointer tidak memiliki referensi valid. Dalam konteks memori, hal tersebut mencegah akses ke alamat yang tidak terdefinisi. Pada baris [8], `bool isEmpty(const Stack* s) {` mendefinisikan fungsi untuk mengecek apakah stack kosong. Parameter bertipe `const` menunjukkan bahwa fungsi ini bersifat read-only dan tidak mengubah isi stack. Di dalam fungsi ini pada baris [9], `return s->top == nullptr;` mengembalikan nilai boolean berdasarkan kondisi pointer `top`. Secara logika, jika `top` tidak menunjuk ke elemen mana pun, maka tidak ada elemen dalam stack. Pembuktian sederhananya yaitu seluruh operasi `push` selalu mengubah `top` dari `nullptr` ke alamat valid, sehingga kondisi `nullptr` hanya mungkin terjadi saat stack kosong. Selanjutnya pada baris [12], `bool isFull(const Stack* s) {` mendefinisikan fungsi untuk mengecek apakah stack penuh. Fungsi tersebut diperlukan karena stack menggunakan array statis dengan ukuran tetap (`MAX`), sehingga tidak dapat berkembang secara dinamis. Pada baris [13], `if (isEmpty(s)) return false;` untuk memastikan bahwa stack kosong tidak dianggap penuh. Kondisi penuh hanya mungkin terjadi jika sudah ada elemen di dalam stack. Baris [14], `return s->top == &(s->data[MAX - 1]);` membandingkan alamat pointer `top` dengan alamat elemen terakhir array. Hal tersebut karena elemen terakhir berada pada indeks `MAX-1`, sehingga pointer `top`

akan menunjuk ke elemen terakhir dalam array. Jika pointer `top` mencapai alamat tersebut, maka tidak ada ruang lagi untuk elemen baru, sehingga stack dinyatakan penuh. Pada baris [17], `void push(Stack* s, int value) {` mendefinisikan operasi `push`, yaitu menambahkan elemen ke stack sesuai prinsip LIFO (Last In First Out). Baris [18], terdapat `if (isFull(s)) throw std::overflow_error("Stack Penuh");` yang berfungsi untuk mencegah overflow. Overflow terjadi jika penambahan elemen dilakukan saat stack sudah mencapai kapasitas maksimum. Tanpa pengecekan ini, pointer dapat melewati batas array dan mengakses memori ilegal yang berbahaya. Baris [20], `if (isEmpty(s)) {` mengecek apakah stack kosong sebelum menentukan posisi penyimpanan elemen pertama. Baris [21], `s->top = &(s->data[0]);` untuk mengatur pointer `top` ke alamat elemen pertama array, karena elemen pertama harus ditempatkan pada indeks awal dan top menunjuk ke base adress array. Pada baris [22] `} else {` dan [23] `s->top++;` digunakan jika stack tidak kosong. Operasi `top++` merupakan pointer arithmetic yang memindahkan pointer ke elemen berikutnya dalam array. Karena array bertipe `int`, maka `top++` secara otomatis menambah alamat sebesar `sizeof(int)` byte untuk memastikan bahwa elemen baru ditempatkan tepat setelah elemen sebelumnya tanpa perlu menghitung indeks secara manual. Pada baris [25], `*(s->top) = value;` menyimpan nilai ke lokasi memori yang ditunjuk oleh pointer `top`. Dereferencing (*) digunakan untuk mengakses isi alamat tersebut. Nilai value ditulis langsung ke array pada posisi yang telah ditentukan oleh pointer. Lalu pada baris [28], `void pop(Stack* s) {` mendefinisikan operasi `pop`, yaitu menghapus elemen teratas dari stack. Baris [29], `if (isEmpty(s)) throw std::underflow_error("Stack Kosong");` untuk mencegah underflow, yaitu kondisi ketika penghapusan dilakukan pada stack kosong. Tanpa pengecekan ini, pointer dapat mengarah ke alamat tidak valid. Pada baris [31], `if (s->top == &(s->data[0])) {` mengecek apakah hanya ada satu elemen dalam stack. Jika `top` berada di indeks 0, maka itu adalah elemen pertama sekaligus terakhir. Baris [32], `s->top = nullptr;` mengembalikan stack ke kondisi kosong setelah elemen terakhir dihapus untuk menjaga invariant bahwa stack kosong selalu memiliki `top == nullptr`. Pada baris [33] `} else {` dan baris [34] `s->top--;` digunakan jika terdapat lebih dari satu elemen. Operasi `top` menggeser pointer ke alamat sebelumnya dalam array. Secara pointer

arithmetic, berarti mengurangi alamat sebesar `sizeof(int)` byte. Hal tersebut secara efektif menghapus elemen teratas karena pointer tidak lagi menunjuk ke elemen tersebut. Pada baris [38], `int peek(const Stack* s) {` mendefinisikan fungsi untuk mengambil nilai elemen teratas tanpa menghapusnya untuk membaca data tanpa mengubah struktur stack. Baris [39], `if (isEmpty(s)) throw std::underflow_error("Stack Kosong");` untuk memastikan bahwa akses hanya dilakukan jika stack tidak kosong, sehingga menghindari dereference pointer yang tidak valid. Baris [40], `return *(s->top);` mengembalikan nilai pada alamat yang ditunjuk oleh pointer `top`. Karena `top` selalu menunjuk ke elemen terakhir yang dimasukkan, maka nilai ini adalah elemen teratas sesuai prinsip LIFO (Last In First Out).

Pada file stack.cpp ini, terdapat beberapa operasi utama yaitu `init`, `isEmpty`, `isFull`, `push`, `pop`, dan `peek`. Seluruh operasi tersebut memiliki kompleksitas waktu O(1) atau konstan. Hal tersebut disebabkan karena setiap operasi hanya melibatkan manipulasi pointer top tanpa adanya perulangan atau proses pencarian data. Contohnya operasi `push` hanya melakukan pengecekan kondisi penuh, kemudian menggeser pointer ke posisi berikutnya dan menyimpan nilai. Begitu pula dengan `pop` yang hanya menggeser pointer ke posisi sebelumnya. Dengan demikian, waktu eksekusi setiap operasi tidak bergantung pada jumlah elemen dalam stack, sehingga dapat disimpulkan bahwa seluruh operasi pada stack bersifat efisien dengan kompleksitas konstan.

# SOAL 2: Si Palui dan Mesin Hitung Aceh

| Time Limit: 1.0 s |
| --- |
| Memory Limit: 64 MB |

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

- sebuah bilangan bulat, atau
- sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

**Batasan**

- $1 \leq n \leq 100$
- $-1000 \leq A_i \leq 1000$

Keterangan:

- $n$ adalah jumlah simbol dalam ekspresi
- $A_i$ adalah nilai bilangan pada input
- Simbol yang valid hanya:
  - bilangan bulat

- operator: +, -, *, /

**Format Input**

n

A1 A2 ... An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang simbol
- Baris kedua berisi simbol A sebanyak n buah

**Format Output**

Sebuah bilangan hasil dari kalkulator Si Palui.

**Contoh Kasus**

| Input | Output |
|---|---|
| 3<br>2 3 + | 5 |
| 9<br>5 1 2 + 4 * + 3 - | 14 |
| 5<br>10 2 / 3 * | 15 |

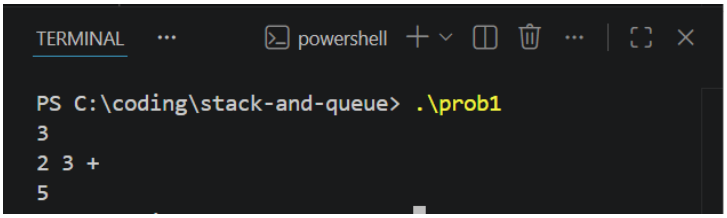
## A. Source Code

*Tabel 2 Source Code prob1.cpp*

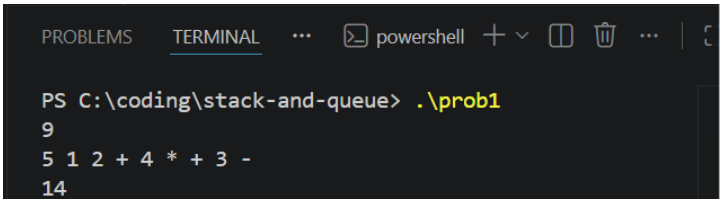
```
#include "stack.h"
#include <iostream>
#include <string>

int main() {
    int n;
    if (!(std::cin >> n)) return 0;

```

```
    Stack s;
    init(&s);

    for (int i = 0; i < n; i++) {
        std::string token;
        std::cin >> token;

        if (isdigit(token[0]) || (token.size() > 1 && token[0] == '-')) {
            push(&s, std::stoi(token));
        } else {
            int val2 = peek(&s); pop(&s);
            int val1 = peek(&s); pop(&s);

            if (token == "+") push(&s, val1 + val2);
            else if (token == "-") push(&s, val1 - val2);
            else if (token == "*") push(&s, val1 * val2);
            else if (token == "/") push(&s, val1 / val2);
        }
    }

    std::cout << peek(&s) << std::endl;
    return 0;
}
```

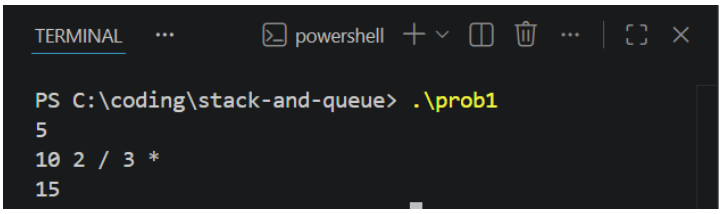
## B. Output Program




*Gambar 1. Screenshot Hasil Jawaban Soal 2A*




*Gambar 2. Screenshot Hasil Jawaban Soal 2B*




*Gambar 3. Screenshot Hasil Jawaban Soal 2C*

## C. Pembahasan

Pada baris [1], syntax `#include "stack.h"` program mengimpor struktur dan fungsi stack yang akan digunakan sebagai media penyimpanan data sementara. Hal ini dilakukan karena perhitungan menggunakan mekanisme LIFO. Cara kerjanya adalah compiler membaca definisi tersebut agar fungsi seperti `push`, `pop`, dan `peek` dapat digunakan. Pada baris [2], `#include <iostream>` digunakan untuk proses input dan output agar program dapat membaca data dan menampilkan hasil. Pada baris [3], `#include <string>` digunakan agar program dapat membaca token dalam bentuk string, karena input dapat berupa angka maupun operator. Selanjutnya ada digunakan tanda kurung kurawal `{}` untuk membatasi blok kode dalam fungsi maupun struktur kontrol seperti percabangan dan perulangan. Selanjutnya pada baris [5], `int main() {` merupakan fungsi utama atau titik awal eksekusi program, tempat seluruh proses dijalankan. Pada baris [6], `int n;` untuk mendeklarasikan variabel `n` yaitu jumlah token yang akan diproses. Pada baris [7], `if (!(std::cin >> n)) return 0;` untuk membaca jumlah `token` sekaligus memastikan input valid. Jika gagal, program dihentikan untuk mencegah error lanjutan. Cara kerjanya adalah dengan memeriksa status stream input, jika gagal, fungsi `main` akan mengembalikan nilai 0 dan mengakhiri eksekusi sebelum terjadi error logika lebih lanjut. Pada baris [9], `Stack s;` membuat variabel `s` bertipe `Stack` yang akan digunakan selama proses perhitungan. Pada baris [10], `init(&s);` untuk mengembalikan `stack` agar

kosong, sehingga pointer `top` berada pada kondisi awal sebelum digunakan. Lalu pada baris [12], `for (int i = 0; i < n; i++) {` melakukan proses perulangan sebanyak jumlah token (`n`) untuk memproses setiap input secara berurutan. Cara kerjanya adalah dengan menggunakan variabel pencacah `i` yang membatasi jalannya logika sesuai dengan jumlah input yang telah didefinisikan sebelumnya. Pada baris [13] `std::string token;` dan baris [14] `std::cin >> token;` digunakan untuk membaca setiap `token` dalam bentuk string. Hal tersebut memungkinkan program membedakan antara angka dan operator. Selanjutnya pada baris [16], `if (isdigit(token[0]) || (token.size() > 1 && token[0] == '-')) {` digunakan untuk mengecek apakah `token` adalah bilangan. Pengecekan karakter pertama dengan `isdigit` mendeteksi angka positif, sedangkan pengecekan `token.size() > 1 && token[0] == '-'` digunakan untuk mengenali bilangan negatif. Mekanismenya adalah memvalidasi struktur string sebelum dilakukan konversi tipe data. Pada baris [17], `push(&s, std::stoi(token));` program mengubah string angka menjadi integer menggunakan fungsi `std::stoi` dan memasukkannya ke dalam `Stack`. Cara kerjanya adalah fungsi `push` akan menaikkan posisi pointer `top` dan menyimpan nilai tersebut ke dalam alamat memori array yang ditunjuk, mengikuti prinsip LIFO. Pada baris [18], `} else {` kondisi yang menandakan bahwa `token` yang dibaca adalah operator. Pada baris [19], `int val2 = peek(&s); pop(&s);` mengambil elemen teratas sebagai operand kedua menggunakan `peek`, lalu menghapusnya dengan `pop`. Cara kerjanya adalah pointer `top` berpindah ke bawah setelah elemen diambil. Baris [20], `int val1 = peek(&s); pop(&s);` mengambil elemen berikutnya sebagai operand pertama. Urutan ini penting agar hasil operasi tetap sesuai terutama untuk pengurangan dan pembagian. Pada baris [22], `if (token == "+") push(&s, val1 + val2);` melakukan operasi penjumlahan, kemudian hasilnya dimasukkan kembali ke stack. Pada baris [23] `else if (token == "-") push(&s, val1 - val2);` , baris [24] `else if (token == "*") push(&s, val1 * val2);` dan baris [25] `else if (token == "/") push(&s, val1 / val2);` program mengecek dan akan mengeksekusi operasi artimatika pengurangan, perkalian dan pembagian jika kondisi benar dengan cara yang sama, yaitu mengambil dua operand, melakukan operasi sesuai operator, lalu memasukkan hasilnya

kembali ke stack. Mekanisme ini memastikan proses berjalan hingga menghasilkan satu nilai akhir. Lalu pada baris [29], `std::cout << peek(&s) << std::endl;` untuk menampilkan hasil akhir yang tersisa di stack yaitu mengambil nilai teratas tanpa menghapusnya. Cara kerjanya adalah fungsi `peek` mengakses alamat yang ditunjuk oleh `top` untuk terakhir kalinya guna mencetak jawaban ke layar pengguna. Pada baris [30], `return 0;` menandakan program selesai dengan normal.

Program pada prob1.cpp digunakan untuk menghitung ekspresi dalam bentuk postfix. Program ini memproses input menggunakan satu perulangan utama yang berjalan sebanyak jumlah token, yaitu `n`. Pada setiap iterasi, program melakukan operasi sederhana seperti membaca input, pengecekan jenis token, serta operasi stack (`push`, `pop`, dan `peek`) yang semuanya memiliki kompleksitas O(1). Oleh karena itu, total kompleksitas waktu program ini adalah O(n) yang menunjukkan bahwa waktu eksekusi program bertambah secara linear terhadap jumlah input, sehingga masih tergolong efisien untuk data berukuran besar.

# SOAL 3: Implementasi Queue

Pada file `queue.h` yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file `queue.cpp`. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan `throw`.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

## A. Source Code

*Tabel 3 Source Code queue.cpp*

```
#include "queue.h"
#include <stdexcept>

void init(Queue* q) {
    q->front = nullptr;
    q->rear = nullptr;
}

bool isEmpty(const Queue* q) {
    return q->front == nullptr;
}

bool isFull(const Queue* q) {
    if (isEmpty(q)) return false;
    return q->rear == &(q->data[MAX - 1]);
}

void enqueue(Queue* q, int value) {
    if (isFull(q)) {
```

```
        throw std::overflow_error("Queue Full");
    }

    if (isEmpty(q)) {
        q->front = &(q->data[0]);
        q->rear = &(q->data[0]);
    } else {
        q->rear++;
    }
    *(q->rear) = value;
}

void dequeue(Queue* q) {
    if (isEmpty(q)) {
        throw std::underflow_error("Queue Empty");
    }

    if (q->front == q->rear) {
        init(q);
    } else {
        q->front++;
    }
}

int front(const Queue* q) {
    if  (isEmpty(q))  throw  std::underflow_error("Queue Empty");
    return *(q->front);
}

int back(const Queue* q) {
```

```
50      if (isEmpty(q)) throw std::underflow_error("Queue Empty");
51      return *(q->rear);
52  }
```

**B. Pembahasan**

Pada baris [1], syntax `#include "queue.h"` mengimpor sttruktur dan deklarasi fungsi queue yang akan digunakan agar implementasi fungsi sesuai dengan definisi yang telah dibuaut sebelumnya. Pada baris [2], `#include <stdexcept>` digunakan untuk menangani error seperti queue penuh (overflow) dan queue kosong (underflow) menggunakan exception. Selanjutnya ada digunakan tanda kurung kurawal `{}` untuk membatasi blok kode dalam fungsi maupun struktur kontrol seperti percabangan dan perulangan. Pada baris [4], `void init(Queue* q) {` mendefinisikan fungsi untuk menginisialisasi queue. Parameter berupa pointer agar perubahan langsung terjadi pada data asli. Pada baris [5] `q->front = nullptr;` dan [6] `q->rear = nullptr;` untuk mengatur queue dalam kondisi kosong. Hal ini dilakukan karena belum ada elemen yang disimpan, sehingga kedua pointer tidak menunjuk ke mana pun. Pada baris [9], `bool isEmpty(const Queue* q) {` mendefinisikan fungsi `isEmpty` dengan parameter `const Queue* q`. Penggunaan `const` memastikan fungsi ini bersifat read-only (hanya membaca). Baris [10], `return q->front == nullptr;` fungsi mengembalikan hasil perbandingan `q->front == nullptr`. Cara kerjanya adalah dengan mengecek apakah pointer `front` tidak menunjuk ke elemen mana pun. Selanjutnya pada baris [13], `bool isFull(const Queue* q) {` mendefinisikan fungsi `isFull` untuk mengecek apakah queue sudah penuh. Baris [14], `if (isEmpty(q)) return false;` untuk memastikan queue kosong tidak dianggap penuh. Baris [15], `return q->rear == &(q->data[MAX - 1]);` untuk mengecek apakah pointer `rear` sudah berada di elemen terakhir array yang menunjukkan bahwa tidak ada ruang lagi untuk menambah elemen. Mekanismenya adalah mengecek apakah pointer belakang sudah mencapai batas fisik tertinggi dari memori array yang dialokasikan. Lalu pada baris [18], `void enqueue(Queue* q, int value) {` yaitu mendefinisikan fungsi untuk

menambahkan elemen ke dalam queue. Pada baris [19], `if (isFull(q)) {` yang berfungsi untuk mengecek apakah queue sudah dalam kondisi penuh sebelum dilakukan penambahan elemen. Cara kerjanya adalah dengan memanggil fungsi `isFull(q)` yang akan mengembalikan nilai boolean. Jika bernilai true, maka program akan masuk ke dalam blok kondisi, yang berarti queue sudah mencapai batas maksimum. Pada baris [20], `throw std::overflow_error("Queue Full");` untuk menghentikan proses penambahan elemen ketika queue sudah penuh dengan cara melempar exception bertipe `overflow_error`. Hal ini dilakukan untuk mencegah penulisan data di luar batas array yang dapat menyebabkan error pada program. Selanjutnya pada baris [23], `if (isEmpty(q)) {` untuk mengecek apakah queue masih kosong sebelum menambahkan elemen pertama. Baris [24] `q->front = &(q->data[0]);` dan [25] `q->rear = &(q->data[0]);` dijalankan ketika queue masih kosong, maka elemen pertama akan ditempatkan pada indeks awal array. Pointer `front` dan `rear` sama-sama diarahkan ke `data [0]` karena elemen pertama sekaligus menjadi awal dan akhir queue. Pada baris [26] `} else {` dan [27] `q->rear++;` dijalankan jika queue tidak kosong, pointer `rear` digeseer ke posisi berikutnya menggunakan `rear++`. Cara kerjanya adalah pointer berpindah ke elemen selanjutnya dalam array, sehingga elemen baru ditempatkan di belakang elemen sebelumya. Pada baris [29], `*(q->rear) = value;` nilai dimasukkan ke dalam queue dengan `*(q->rear) = value`. Cara kerjanya adalah dengan mengakses alamat yang ditunjuk pointer `rear`, kemudian menyimpan nilai pada posisi tersebut. Berikutnya pada baris [32], `void dequeue(Queue* q) {` didefinisikan fungsi `dequeue` yang digunakan untuk menghapus elemen dari bagian depan queue. Baris [33] `if (isEmpty(q)) {` dan [34] `throw std::underflow_error("Queue Empty");` yaitu pengecekan `isEmpty` dilakukan untuk memastikan bahwa queue tidak kosong sebelum penghapusan. Jika kosong, program akan melempar exception `underflow_error` untuk mencegah akses data yang tidak valid. Lalu pada baris [37] `if (q->front == q->rear) {` dan [38] `init(q);` merupakan kondisi yang jika hanya terdapat satu elemen (`front == rear`), maka fungsi `init` dipanggil untuk mengosongkan queue. Cara kerjanya adalah dengan mengembalikan pointer ke `nullptr`, sehingga queue kembali ke kondisi awal. Pada baris [39] `} else {` dan [40] `q-`

`>front++;` yang mengecek jika lebih dari satu elemen, pointer `front` digeser ke depan menggunakan `front++`. Hal ini membuat elemen pertama tidak lagi diakses, sehingga dianggap telah dihapus. Pada baris [44], `int front(const Queue* q) {` mendefinisikan fungsi untuk mengambil nilai elemen paling depan tanpa menghapusnya. Menggunakan pointer bertipe `const` agar fungsi hanya membaca data tanpa mengubah isi `queue`. Cara kerjanya adalah dengan mengakses elemen yang ditunjuk oleh pointer `front`. Pada baris [45], `if (isEmpty(q)) throw std::underflow_error("Queue Empty");` untuk memastikan bahwa queue tidak kosong sebelum diakses untuk mencegah pengambilan data dari kondisi yang tidak valid. Cara kerjanya dengan memanggil fungsi `isEmpty`, dan jika bernilai true maka program akan melempar error sehingga proses dihentikan. Pada baris [46], `return *(q->front);` mengembalikan nilai yang berada di posisi depan queue. Cara kerjanya adalah pointer `front` menunjuk ke elemen pertama dalam array, kemudian dilakukan dereference (*) untuk mengambil nilai yang tersimpan pada alamat tersebut. Lalu pada baris [49], `int back(const Queue* q) {` mendefinisikan fugsi untuk mengambil nilai elemen paling belakang dari queue tanpa menghapusnya dan digunakan untuk melihat elemen terakhir yang masuk. Pada baris [50], `if (isEmpty(q)) throw std::underflow_error("Queue Empty");` berfungsi untuk memastikan bahwa `queue` tidak dalam kondisi kosong sebelum mengambil data. Hal ini dilakukan untuk mencegah kesalahan akses pada data yang tidak ada. Cara kerjanya dengan memanggil fungsi `isEmpty(q)`, jika bernilai true, maka program akan melempar exception `underflow_error` dan menghentikan proses, sehingga tidak terjadi akses ke alamat memori yang tidak valid. Baris [51], `return *(q->rear);` untuk mengembalikan nilai yang berada di posisi belakang queue. Cara kerjanya adalah pointer `rear` menunjuk ke elemen terakhir, lalu dilakukan dereference untuk mengambil nilainya tanpa mengubah isi queue.

Pada queue.cpp, operasi yang tersedia meliputi `init`, `isEmpty`, `isFull`, `enqueue`, `dequeue`, `front`, dan `back`. Sama seperti stack, seluruh operasi pada queue juga memiliki kompleksitas waktu O(1). Hal ini dikarenakan setiap operasi hanya melibatkan perubahan posisi pointer `front` dan `rear` tanpa melakukan iterasi terhadap elemen-elemen dalam queue. Misalnya, pada operasi `enqueue`, program hanya menambahkan elemen di

bagian belakang dengan menggeser pointer `rear`, sedangkan `dequeue` hanya menggeser pointer `front` untuk menghapus elemen. Karena tidak ada proses perulangan, maka waktu yang dibutuhkan untuk setiap operasi tetap konstan, sehingga queue juga termasuk struktur data yang efisien dalam waktu.

# SOAL 4: Si Palui dan Deretan Angka

| Time Limit: 1.0 s |
| --- |
| Memory Limit: 64 MB |

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah $k$ angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih $k$ angka berurutan. Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

**Batasan**

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq A_i \leq 1000$

Keterangan:

- $n$: jumlah elemen
- $k$: ukuran jendela
- $A_i$: elemen array

**Format Input**

n k

A1 A2 ... An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok yang dibentuk
- Baris kedua berisi simbol A sebanyak n buah

**Format Output**

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi.

**Contoh Kasus**

| Input | Output |
|---|---|
| 5 3<br>1 2 3 4 5 | 6 9 12 |
| 5 5<br>1 2 3 4 5 | 15 |
| 6 2<br>4 -1 2 -3 5 1 | 3 1 -1 2 6 |

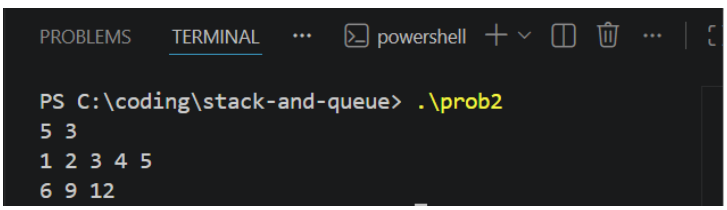
## A. Source Code

*Tabel 4 Source Code prob2.cpp*

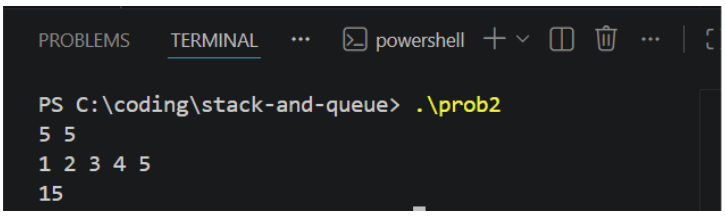
```
#include "queue.h"
#include <iostream>

int main() {
    int n, k;
    if (!(std::cin >> n >> k)) return 0;

    int A[MAX];
    for (int i = 0; i < n; i++) {
        std::cin >> A[i];
    }

    for (int i = 0; i <= n - k; i++) {
        int current_sum = 0;
        Queue q;
        init(&q);

```

```
        for (int j = i; j < i + k; j++) {
            enqueue(&q, A[j]);
            current_sum += A[j];
        }

        std::cout << current_sum << (i == n - k ? "" : " ");
    }
    std::cout << std::endl;

    return 0;
}
```

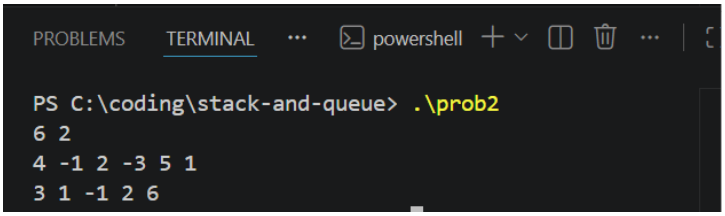
## B. Output Program




*Gambar 4. Screenshot Hasil Jawaban Soal 4A*




*Gambar 5. Screenshot Hasil Jawaban Soal 4B*




*Gambar 6. Screenshot Hasil Jawaban Soal 4C*

## C. Pembahasan

Pada baris [1], syntax `#include "queue.h"` berfungsi untuk mengimpor struktur dan fungsi queue yang akan digunakan dalam program. Pada baris [2], `#include <iostream>` digunakan untuk proses input dan output, yaitu membaca data dan menampilkan hasil ke layar. Selanjutnya ada digunakan tanda kurung kurawal `{}` untuk membatasi blok kode dalam fungsi maupun struktur kontrol seperti percabangan dan perulangan. Berikutnya baris [4], `int main() {` merupakan titik awal atau fungsi utama eksekusi program. Baris [5], `int n, k;` mendeklarasikan dua variabel, yaitu `n` sebagai jumlah data dan `k` sebagai ukuran window (jumlah elemen yang akan dijumlahkan setiap iterasi). Pada baris [6], `if (!(std::cin >> n >> k)) return 0;` membaca input `n` dan `k`, sekaligus memastikan input valid. Jika gagal, program dihentikan untuk mencegah error. Pada baris [8], `int A[MAX];` mendeklarasikan array untuk menyimpan data input. Pada baris [9], `for (int i = 0; i < n; i++) {` merupakan perulangan yang digunakan untuk membaca data sebanyak `n` kali. Hal ini dilakukan karena jumlah data yang akan dimasukkan sudah diketahui melalui variabel `n`. Cara kerjanya adalah variabel `i` dimulai dari 0 dan akan bertambah satu setiap iterasi hingga mencapai `n-1`, sehingga seluruh indeks array dapat diakses secara berurutan. Pada baris [10], `std::cin >> A[i];` berfungsi untuk membaca input dari pengguna dan menyimpannya ke dalam array `A` pada indeks ke-`i`. Hal ini dilakukan agar setiap nilai yang dimasukkan dapat disimpan dan digunakan dalam proses selanjutnya. Cara kerjanya adalah `std::cin` mengambil satu nilai input, kemudian menyimpannya langsung ke posisi array yang sesuai dengan indeks iterasi saat itu. Berikutnya pada baris [13], `for (int i = 0; i <= n - k; i++) {` digunakan untuk menggeser window sepanjang array. Hal tersebut dilakukan agar setiap kelompok berisi `k` elemen dapat diproses. Batas `n - k` memastikan tidak terjadi akses di luar array. Baris [14], `int current_sum = 0;` digunakan untuk menyimpan jumlah sementara dari setiap window. Pada baris [15], `Queue q;` mendeklarasikan `queue` untuk menampung elemen satu window. Baris [16], `init(&q);` menginisialisasi `queue` agar kosong sebelum digunakan pada setiap iterasi. Selanjutnya pada baris [18], `for (int j = i; j < i + k;` merupakan perulangan yang digunakan untuk mengambil sejumlah `k` elemen berturut-turut dari array, dimulai dari indeks `i`. Cara kerjanya adalah variabel `j`

dimulai dari `i` dan akan bertambah hingga `i + k - 1`, sehingga dapat diakses secara berurutan tanpa melewati batas window. Baris [19], `enqueue(&q, A[j]);` berfungsi untuk memasukkan elemen array ke dalam `queue` agar elemen dalam satu window dapat dikelola secara terstruktur. Cara kerjanya adalah nilai `A[j]` dimasukkan ke bagian belakang `queue` menggunakan fungsi `enqueue`, sehingga urutan elemen tetap terjaga sesuai dengan urutan dalam array. Baris [20], `current_sum += A[j];` digunakan untuk menambahkan nilai elemen ke dalam variabel `current_sum` untuk menghitung total dari setiap window. Cara kerjanya yaitu setiap elemen yang diakses pada iterasi akan langsung dijumlahkan secara bertahap, sehingga setelah perulangan selesai, `current_sum` berisi jumlah seluruh elemen dalam window tersebut. Selanjutnya pada baris [23], `std::cout << current_sum << (i == n - k ? "" : " ");` untuk menampilkan hasil penjumlahan untuk setiap window. Cara kerjanya adalah mencetak nilai `current_sum`, dan menambahkan spasi jika belum mencapai iterasi terakhir. Lalu pada baris [25], `std::cout << std::endl;` digunakan untuk membuat baris baru setelah seluruh hasil ditampilkan. Cara kerjanya `std::endl` mengirim karakter newline ke output dan memindahkan posisi kursor ke baris berikutnya. Baris [27]. `return 0;` Menandakan program selesai dengan normal atau sukses.

Program pada prob2.cpp ini digunakan untuk menghitung jumlah elemen dalam setiap kelompok (window) dengan ukuran tertentu. Program ini menggunakan dua perulangan bersarang, yaitu perulangan luar untuk menentukan posisi awal window, dan perulangan dalam untuk menjumlahkan elemen dalam window tersebut. Perulangan luar berjalan sebanyak kurang lebih n, sedangkan perulangan dalam berjalan sebanyak k untuk setiap iterasi. Oleh karena itu, total kompleksitas waktu program ini adalah $O(n \times k)$ yang terjadi karena setiap window dihitung ulang dari awal tanpa memanfaatkan hasil perhitungan sebelumnya. Meskipun pendekatan ini sederhana dan mudah dipahami, namun kurang efisien jika dibandingkan dengan metode sliding window yang lebih optimal.

# TAUTAN GIT

https://github.com/DSA26-ULM/module-2-stack-and-queue-Salma-NL.git